DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (I/2026)

Guía de ejercicios: Computador básico sin saltos

Ayudantes: Alberto Maturana (alberto.maturana@uc.cl), Mario Rojas (mario.denzel@estudiante.uc.cl, Nicolás Romo Aubele (nroma@uc.cl))

Pregunta 1: Preguntas conceptuales

1. Explique la diferencia entre las instrucciones `MOV A,var` y `MOV A,(var)`.

2. Un *assembler* es una herramienta de software que traduce un programa escrito en *assembly* a su representación equivalente en código máquina (binario), que puede ser ejecutado directamente por el procesador. Describa en detalle los pasos que debe realizar un *assembler* para procesar código *assembly* del computador básico.

3. Un *disassembler* es un programa que transforma código binario ejecutable en *assembly*. Describa cómo funcionaría un *disassembler* para el computador básico. ¿Es posible obtener el *assembly* original a partir de un programa ubicado en la memoria de instrucciones?

Solución:

1. En primer lugar es importante notar que ambas instrucciones válidas en nuestro set, la diferencia entre ellas radica en que la primera nos dice que movamos a A **el valor de la dirección** `var`, esto quiere decir que no estamos moviendo el valor de la variable `var` si no dónde está guardada en la memoria de datos. Por otra parte, la segunda instrucción si nos está guardando el valor de la variable `var` en el registro A, dado que estamos pidiendo el valor al que apunta la variable.

2. El primer paso es registrar todos los *labels* definidos en el segmento `DATA`. Luego, en el segmento `CODE`, sustituir las referencias a estos *labels* por el literal correspondiente (dir. de memoria). Finalmente, se debe transformar cada instrucción del segmento `CODE` a un *opcode*.

3. Un *disassembler* para el computador básico trabajará tomando cada uno de los *opcodes* en las correspondientes instrucciones. Esto es posible gracias a que los *opcodes* tienen una estructura fija y que cada uno corresponde a solo una instrucción del *assembly*. En el caso de las instrucciones que usan 2 *opcodes*, el *disassembler* deberá ser capaz de reconocer esta secuencia. Los literales son también sencillos de reconocer, dada la estructura de los *opcodes*. Finalmente, a partir de un programa ubicado en la memoria de instrucciones, no es posible reconstruir el código *assembly* original, ya que los *labels*

son traducidos como direcciones (números), perdiendo la información sobre sus nombres explícitos. Sin embargo, el código reconstruido funcionaría de igual manera que el código original, solo que los nombres de las variables y *labels* serían distintos.

Pregunta 2: *Assembly*: Análisis y depuración de código

a) (**I1 2024-2**) Describa qué hace el fragmento (a) escrito en el *Assembly* del computador básico de 8 *bits* visto en el curso. Indique los valores de los registros `A` y `B` al finalizar la ejecución del código.

```
DATA:
    var1 3
    var2 5
CODE:
    MOV B,254
    MOV A,B
    NOT A,A
    MOV B,A
    MOV A,(B)
```

b) Dado el siguiente código, indique el valor final de la variable `res`. Muestre la ejecución de este paso a paso.

```
DATA:
    ptr1  2
    ptr2  4
    v0    10
    v1    20
    v2    30
    v3    40
    res   0

CODE:
    MOV B, (ptr1)
    MOV A, (B)
    MOV (res), A
    MOV B, (ptr2)
    MOV A, (B)
    ADD A, (res)
    MOV (res), A
```

c) Dado el siguiente código, indique el valor final de la variable `res`. Muestre la ejecución de este paso a paso.

```
DATA:
    arr0  3
    arr1  7
    arr2  2
    arr3  5
    res   0
CODE:
    MOV B, 0
    MOV A, (B)
    MOV (res), A
    INC B
    MOV A, (B)
    ADD A, (res)
    MOV (res), A
    INC B
    MOV A, (B)
    ADD A, (res)
    MOV (res), A
    INC B
    MOV A, (B)
    ADD A, (res)
    MOV (res), A
```

Solución:

a) El fragmento obtiene el valor `NOT B` y lo utiliza como dirección de memoria para guardar el dato almacenado en ella en el registro `A`. Dado que `B` almacena el valor 254 de 8 *bits*, se tiene que `NOT B = 1`. Luego, como las variables se almacenan desde la dirección 0 de la memoria de datos, se tiene que `A = Mem[1] = var2`. Por lo tanto, se tiene que `A = 5` y `B = 1`.

```
DATA:
    var1 3
    var2 5
CODE:
    MOV B,254   // B = 254
    MOV A,B     // A = 254
    NOT A,A     // A = 1
    MOV B,A     // B = 1
    MOV A,(B)   // A = 5
```

b) Para resolver este ejercicio es fundamental tener en cuenta el mapa de memoria, ya que las variables se almacenan en orden secuencial desde la dirección `0`. De esta forma, `ptr1` se encuentra en la dirección `0` con valor `2`, `ptr2` en la dirección `1` con valor `4`, y los valores `v0` a `v3` en las direcciones `2` a `5` respectivamente.

```
DATA:
    ptr1  2
    ptr2  4
    v0    10
    v1    20
    v2    30
    v3    40
    res   0

CODE:
    MOV B, (ptr1)       // B = 2
    MOV A, (B)          // MOV A, (2) -> A = 10
    MOV (res), A        // res = 10
    MOV B, (ptr2)       // B = 4
    MOV A, (B)          // MOV A, (4) -> A = 30
    ADD A, (res)        // A = 30 + 10
    MOV (res), A        // res = 40
```

c) El código comienza cargando `0` en `B` directamente, usando `B` como puntero a la primera posición del arreglo. Con `MOV A, (B)` se carga el contenido de la dirección `0`, es decir `arr0 = 3`, que se guarda en res como valor inicial. A partir de aquí el patrón se repite tres veces: `INC B` avanza el puntero a la siguiente dirección, `MOV A, (B)` carga el elemento correspondiente, y `ADD A, (res)` lo acumula al resultado guardado. De esta forma el código recorre secuencialmente los cuatro elementos del arreglo sumándolos uno a uno.

```
DATA:
    arr0  3
    arr1  7
    arr2  2
    arr3  5
    res   0
CODE:
    MOV B, 0         // B = 0
    MOV A, (B)       // MOV A, (0) -> A = 3
    MOV (res), A     // res = 3
    INC B            // B = 1
    MOV A, (B)       // MOV A, (1) -> A = 7
    ADD A, (res)     // A = 7 + 3 = 10
    MOV (res), A     // res = 10
    INC B            // B = 2
    MOV A, (B)       // MOV A, (2) -> A = 2
    ADD A, (res)     // A = 2 + 10 = 12
    MOV (res), A     // res = 12
    INC B            // B = 3
    MOV A, (B)       // MOV A, (3) -> A = 5
    ADD A, (res)     // A = 5 + 12 = 17
    MOV (res), A     // res = 17
```

Pregunta 3: Probelma en el laboratorio

a) Has sido contratado para optimizar el sistema de adquisición de datos de un laboratorio de bioingeniería. El laboratorio cuenta con una cámara de guantes utilizada para manipular sustancias peligrosas, la cual monitorea continuamente dos variables del ambiente interior: temperatura (en un rango de 0° a 15°C) y humedad relativa (de 0 % a 15 %).

Dado que ambas variables tienen un rango de solo 16 valores posibles, cada una puede representarse con exactamente 4 *bits*. Sin embargo, el canal de comunicación entre la cámara y el computador central que monitoriza el sistema tiene un ancho de banda de 8 *bits* por transmisión, y el protocolo exige que **ambas variables se envíen en un único paquete**, es decir, no es válido transmitir primero la temperatura y luego la humedad por separado.

Los técnicos del laboratorio tienen ambos valores almacenados individualmente en memoria, pero no saben cómo combinarlos en un solo *byte* para su transmisión. Los técnicos, además te comentan que ambos sensores presentan problemas y algunas veces presentan *bits* sucios en sus mediciones, por lo que quieren asegurarse que las mediciones enviadas sean correctas. Escribe un programa en *Assembly* del computador básico que empaquete ambos valores en una sola variable `sensor_data`.

Hint: Investiga acerca de *nibbles* y máscaras de *bits*.

b) El equipo del laboratorio te comenta que el sistema implementado en los sensores es un éxito y envía los valores de forma correcta al computador central. Sin embargo, se dieron cuenta de que este último no está obteniendo las mediciones de forma correcta, y te piden que analices el siguiente código para identificar los errores:

```
DATA:
    sensor_data  10100101
    temp         00000000
    hum          00000000
CODE:
    MOV A, sensor_data
    OR A, 0x0F
    MOV (hum), A
    MOV A, (sensor_data)
    AND A, 0xFF
    SHL A, A
    SHL A, A
    SHL A, A
    MOV temp, A
```

Solución:

El problema se resuelve en dos etapas. En la primera se procesa el valor de `temp`, y en la segunda el de `hum`, combinándolos finalmente en `sensor_data`. En la primera etapa se carga `temp` en el registro `A` y se aplica la máscara `0x0F` mediante un `AND`. Esto es una medida de sanitización: aunque `temp` debería tener sus 4 *bits* altos en cero, no podemos asumirlo, por lo que nos aseguramos de que solo los 4 *bits* bajos estén activos antes de continuar. Luego se realizan 4 desplazamientos a la izquierda con `SHL` para llevar el *nibble* bajo hacia los 4 *bits* altos, dejando los 4 *bits* bajos en cero. El resultado se guarda temporalmente en `sensor data`.

En la segunda etapa se carga `hum` en `A` y se le aplica la misma máscara `0x0F`, esta vez para garantizar que el *nibble* alto de `hum` esté limpio y no interfiera con `temp` al momento de combinarlos. Luego se carga el valor temporal de `sensor_data` en `B`, que contiene `temp` ya desplazado. Finalmente se aplica `OR A, B`: como ambos valores tienen ceros en los *bits* donde el otro tiene datos, el `OR` los combina sin conflicto, obteniendo el *byte* empaquetado final que se guarda en `sensor`$_d ata$.

El código se presenta a continuación

```
DATA:
    temp         00001010
    hum          00000101
    sensor_data  0
CODE:
    // Llevar temp al nibble alto
    MOV A, (temp)          // A = 00001010
    AND A, 0x0F            // A = 00001010 Sanitizamos (sólo por precaución)
    SHL A, A               // A = 00010100
    SHL A, A               // A = 00101000
    SHL A, A               // A = 01010000
    SHL A, A               // A = 10100000
    MOV (sensor_data), A   // (sensor_data) = 10100000 Guardamos el valor de la temperatura

    // Sanitizar hum y combinar
    MOV A, (hum)           // A = 00000101
    AND A, 0x0F            // A = 00000101 Sanitizamos para aseguramos que el nibble alto sea 0.
    MOV B, (sensor_data)   // B = 10100000
    OR A, B                // A = 10100101 // Ambos valores están en un byte
    MOV (sensor_data), A   // sensor_data = 10100101
```

b) El código presenta cinco errores. El primero se encuentra en la instrucción `MOV A, sensor data`, donde se omiten los paréntesis, por lo que en lugar de cargar el contenido de `sensor data` en `A`, se está cargando su dirección de memoria. Debe corregirse a `MOV A, (sensor_data)`.

El segundo error está en el uso de `OR A, 0x0F` para extraer el *nibble* bajo. La operación `OR` fuerza a 1 todos los *bits* cubiertos por la máscara en vez de aislarlos, por lo que el resultado siempre sería `XXXX1111` independiente del valor original. La operación correcta es `AND A, 0x0F`, que conserva únicamente los 4 *bits* bajos y apaga el resto.

El tercer error es la máscara utilizada para extraer el nibble alto: `0xFF` equivale a `11111111`, lo que no filtra ningún *bit* y deja `sensor_data` completamente intacto en `A`. La máscara correcta es `0xF0 (11110000)`, que aísla únicamente los 4 bits altos.

El cuarto error tiene dos componentes: se usa `SHL` en lugar de `SHR`, y se realizan solo 3 desplazamientos en vez de 4. Para normalizar el *nibble* alto hacia los *bits* bajos es necesario desplazar hacia la derecha, y como el *nibble* ocupa las posiciones b7–b4, se requieren exactamente 4 desplazamientos para llevarlo a b3–b0.

Finalmente, el quinto error se encuentra en `MOV temp, A`, donde al igual que en el primer error se omiten los paréntesis, escribiendo en la dirección literal de `temp` en vez de en su contenido. Debe corregirse a `MOV (temp), A`.

El código corregido se presenta a continuación:

```
DATA:
    sensor_data  10100101
    temp         0
    hum          0
CODE:
    // Extraer humedad
    MOV A, (sensor_data)   // A = 10100101
    AND A, 0x0F            // A = 00000101
    MOV (hum), A           // hum = 00000101

    // Extraer nibble alto (temperatura) y normalizar
    MOV A, (sensor_data)   // A = 10100101
    AND A, 0xF0            // A = 10100000
    SHR A, A               // A = 01010000
    SHR A, A               // A = 00101000
    SHR A, A               // A = 00010100
    SHR A, A               // A = 00001010
    MOV (temp), A          // temp = 00001010
```

Pregunta 4: Modificaciones al computador básico

A partir del diagrama del computador básico visto en clases, realice las modificaciones necesarias a la arquitectura para ejecutar las siguientes instrucciones e indique la combinación de señales que deben tener para ser ejecutadas en un ciclo de reloj:

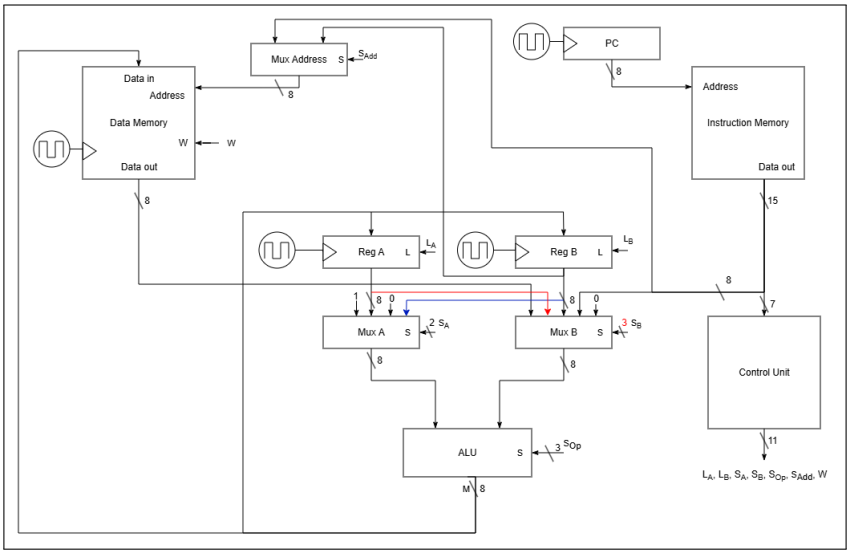
a) `SADD A,A`: Suma el valor de `A` consigo mismo y lo guarda en el registro `A`.

b) Agregar un tercer registro `C`, que sirva tanto para acumular resultados de la `ALU` como para realizar direccionamiento indirecto.

c) Basado en el inciso anterior, implemente las modificaciones necesarias para que el registro `C` mantenga sus funcionalidades, pero que además sea capaz de almacenar un valor de la memoria de datos o de la `ALU` de manera persistente.

Solución:

a) En esta pregunta, se pide agregar una instrucción que nos permita operar dos veces son el valor de `A` en el mismo ciclo, para abordar esto se debe notar que la `ALU` no puede recibir el valor de `A` en sus dos entradas en el mismo ciclo, por lo tanto, para realizar esta modificación nos centraremos en los muxs de selección de datos de entrada a la `ALU`. La solución que se presenta es más general debido a que permite realizar lo mismo con `B` también.

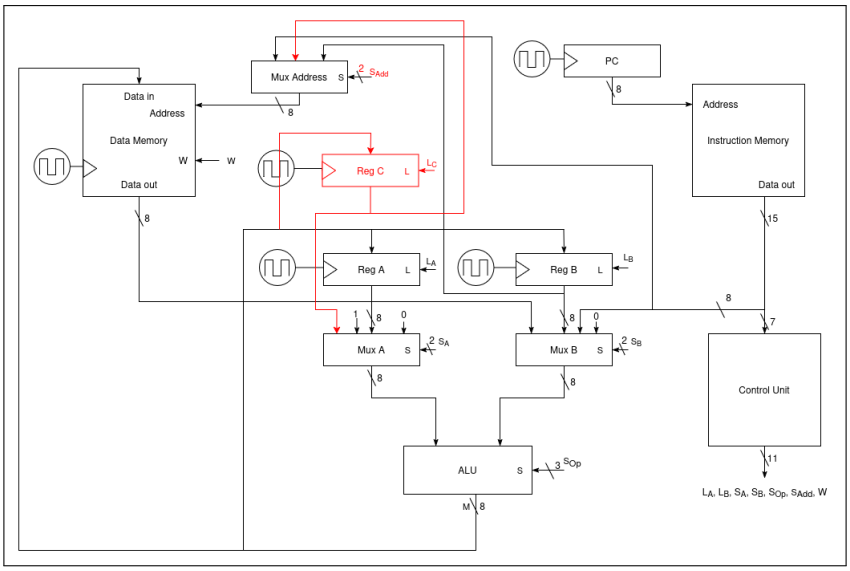


La tabla de señales para la instrucción descrita es la siguiente:

Instrucción	Operandos	Condición	L_A	L_B	S_A0,1	S_B0,1	S_{Op}0,1,2	S_{Add}0,1	W
ADD	A, A	-	1	0	A	A	ADD	-	0

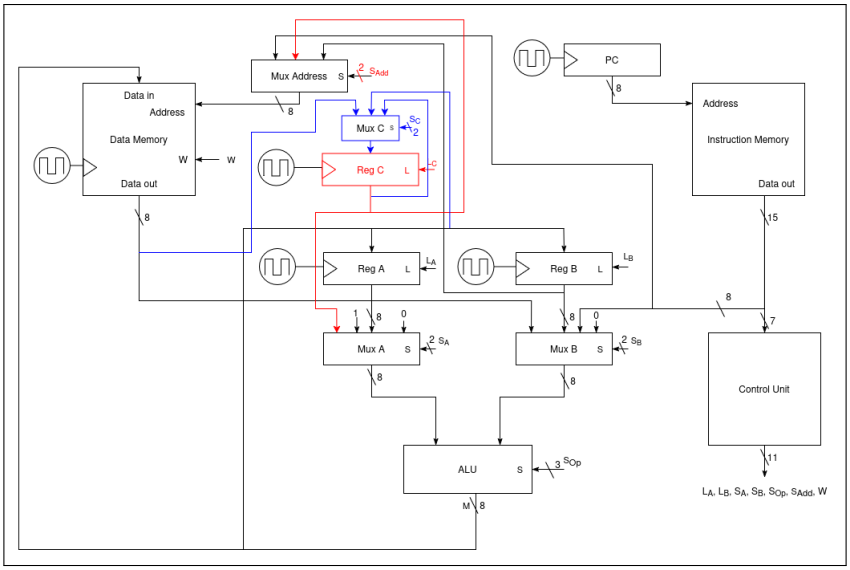
b) Para este inciso se pide implementar un nuevo registro `C`, el cual al igual que los registros `A` y `B` debe estar conectado a la señal de *clock* para que se encuentre en sincronía con el resto

de la arquitectura. Por otra parte, como se pide que cumpla la función de almacenador de resultados, entonces se debe conectar la salida de la `ALU` a este registro. Por último, para operar con sus valores se deben realizar dos conexiones: al espacio libre del `MUX A` y al `MUX Address`. Es importante notar que la señal de selección del `MUX Address` nuevamente crece en un *bit*, mientras que la del `MUX A` no crece en tamaño.



Instrucción	Operandos	Condición	L_A	L_B	L_C	S_A0,1	S_B0,1	S_{Op}0,1,2	S_{Add}0,1	W
MOV	(C), B	-	0	1	0	ZERO	B	ADD	C	1
ADD	C, A, B	-	0	0	1	A	B	ADD	-	0

c) Este ejercicio complementa al anterior, en este caso, debemos poder tener una forma de seleccionar lo que entra en el registro `C`, para este ejercicio en particular este debe tener 3 posibles entradas: La salida de la memoria de instrucciones, la salida de la `ALU` (para no perder la funcionalidad implementada en b) y la salida del registro `C` (para almacenar su propio valor sin pasar por la `ALU`). El diagrama de estas modificaciones se ve de la siguiente manera:



Es importante notar que la instrucción puede ser ejecutada por las dos siguietes combinaciones de señales.

Instrucción	Operandos	Condición	L_A	L_B	L_C	S_A0,1	S_B0,1	S_{Op}0,1,2	S_{Add}0,1	W	S_C
MOV	C, (dir)	-	0	0	1	-	-	-	LIT	0	DOUT
MOV	C, (dir)	-	0	0	1	ZERO	DOUT	ADD	LIT	0	ALU

Tabla de instrucciones

Instrucción	Operandos	*Opcode*	Condición	L_A	L_B	S_A0,1	S_B0,1	S_{Op}0,1,2	S_{Add}0,1	W
MOV	A,B	0000000		1	0	ZERO	B	ADD	-	0
	B,A	0000001		0	1	A	ZERO	ADD	-	0
	A,Lit	0000010		1	0	ZERO	LIT	ADD	-	0
	B,Lit	0000011		0	1	ZERO	LIT	ADD	-	0
	A,(Dir)	0000100		1	0	ZERO	DOUT	ADD	LIT	0
	B,(Dir)	0000101		0	1	ZERO	DOUT	ADD	LIT	0
	(Dir),A	0000110		0	0	A	ZERO	ADD	LIT	1
	(Dir),B	0000111		0	0	ZERO	B	ADD	LIT	1
	A,(B)	0001000		1	0	ZERO	DOUT	ADD	B	0
	B,(B)	0001001		0	1	ZERO	DOUT	ADD	B	0
	(B),A	0001010		1	0	A	ZERO	ADD	B	1
ADD	A,B	0001011		1	0	A	B	ADD	-	0
	B,A	0001100		0	1	A	B	ADD	-	0
	A,Lit	0001101		1	0	A	LIT	ADD	-	0
	A,(Dir)	0001110		1	0	A	DOUT	ADD	LIT	0
	A,(B)	0001111		1	0	A	DOUT	ADD	B	0
	(Dir)	0010000		0	0	A	B	ADD	LIT	1
SUB	A,B	0010001		1	0	A	B	SUB	-	0
	B,A	0010010		0	1	A	B	SUB	-	0
	A,Lit	0010011		1	0	A	LIT	SUB	-	0
	A,(Dir)	0010100		1	0	A	DOUT	SUB	LIT	0
	A,(B)	0010101		1	0	A	DOUT	SUB	B	0
	(Dir)	0010110		0	0	A	B	SUB	LIT	1
AND	A,B	0010111		1	0	A	B	AND	-	0
	B,A	0011000		0	1	A	B	AND	-	0
	A,Lit	0011001		1	0	A	LIT	AND	-	0
	A,(Dir)	0011010		1	0	A	DOUT	AND	LIT	0
	A,(B)	0011011		1	0	A	DOUT	AND	B	0
	(Dir)	0011100		0	0	A	B	AND	LIT	1
OR	A,B	0011101		1	0	A	B	OR	-	0
	B,A	0011110		0	1	A	B	OR	-	0
	A,Lit	0011111		1	0	A	LIT	OR	-	0
	A,(Dir)	0100000		1	0	A	DOUT	OR	LIT	0
	A,(B)	0100001		1	0	A	DOUT	OR	B	0
	(Dir)	0100010		0	0	A	B	OR	LIT	1
NOT	A,A	0100011		1	0	A	-	NOT	-	0
	B,A	0100100		0	1	A	-	NOT	-	0
	(Dir)	0100101		0	0	A	B	NOT	LIT	1
XOR	A,B	0100110		1	0	A	B	XOR	-	0
	B,A	0100111		0	1	A	B	XOR	-	0
	A,Lit	0101000		1	0	A	LIT	XOR	-	0
	A,(Dir)	0101001		1	0	A	DOUT	XOR	LIT	0
	A,(B)	0101010		1	0	A	DOUT	XOR	B	0
	(Dir)	0101011		0	0	A	B	XOR	LIT	1
SHL	A,A	0101100		1	0	A	-	SHL	-	0
	B,A	0101101		0	1	A	-	SHL	-	0
	(Dir)	0101110		0	0	A	B	SHL	LIT	1
SHR	A,A	0101111		1	0	A	-	SHR	-	0
	B,A	0110000		0	1	A	-	SHR	-	0
	(Dir)	0110001		0	0	A	B	SHR	LIT	1
INC	B	0110010		0	1	ONE	B	ADD	-	0
	(B)	0110011		0	0	ONE	DOUT	ADD	B	1
	(Dir)	0110100		0	0	ONE	DOUT	ADD	LIT	1

Tabla 1: ISA del computador básico.